

SCC-0570 - Capítulo 4 Perceptron de Camada Única e Adaline

João Luís Garcia Rosa[1]

[1]Instituto de Ciências Matemáticas e de Computação
Universidade de São Paulo - São Carlos

2019

Sumário

1. Perceptron
   - Histórico
   - O perceptron
   - Convergência do perceptron
2. Adaline
   - Adaptive Linear Element

Sumário

1. Perceptron
   - Histórico
   - O perceptron
   - Convergência do perceptron
2. Adaline
   - Adaptive Linear Element

História

- Nos primórdios das RNA (1943-1958), vários pesquisadores contribuíram:
  - McCulloch e Pitts (1943) introduziram a ideia de redes neurais como máquinas computacionais [4].
  - Hebb (1949) postulou a primeira regra para aprendizado auto-organizado [3].
  - Rosenblatt (1958) propôs o perceptron como o primeiro modelo para aprendizado com professor (supervisionado) [7].
- O perceptron é a forma mais simples de uma RNA usada para classificar padrões *linearmente separáveis*.
- Basicamente, consiste de um único neurônio com pesos sinápticos e bias ajustáveis.
- A prova de convergência do procedimento de aprendizado proposto por Rosenblatt é conhecida como teorema de convergência do perceptron.

História

- O perceptron construído em torno de um único neurônio é limitado a realizar classificação de padrões com apenas duas classes (hipóteses).
- Expandindo a camada de saída do perceptron para incluir mais que um neurônio, pode-se classificar mais de duas classes.
- Entretanto, essas classes devem ser linearmente separáveis para o perceptron funcionar: superfície de decisão tem a forma de um hiperplano entre as duas classes.
- A regra de decisão é designar x à classe $\mathscr{C}_1$ se a saída for $y = +1$ e à classe $\mathscr{C}_2$ se a saída for -1.
- Existem duas regiões separadas pelo hiperplano: $\sum w_i x_i - \theta = 0$.
- Se o espaço for o $\mathbb{R}^2$, a região de separação é uma reta.

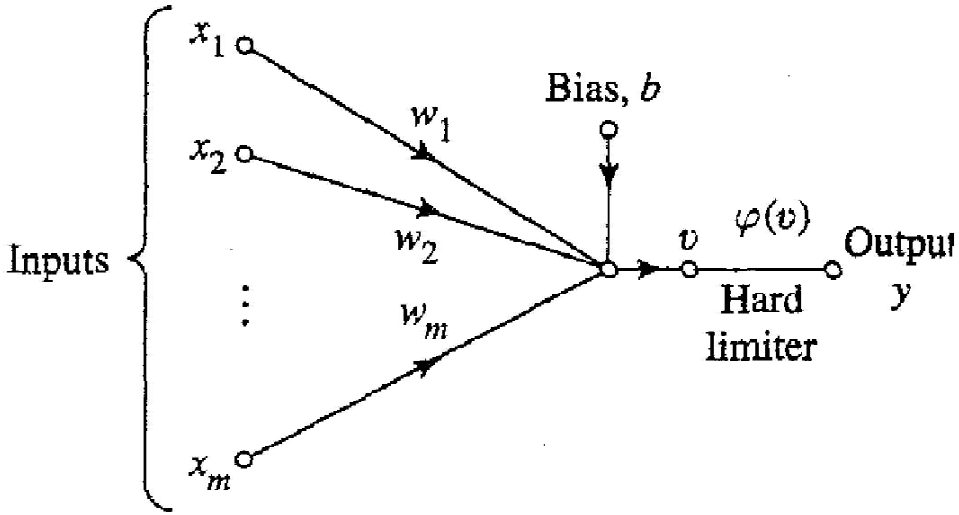
Sumário

1. Perceptron
   - Histórico
   - O perceptron
   - Convergência do perceptron
2. Adaline
   - Adaptive Linear Element

Modelo de McCulloch-Pitts

- O perceptron é construído em volta de um neurônio não linear, o modelo de McCulloch-Pitts [2].

$$v = \sum_{i=1}^{m} w_i x_i + b \tag{1}$$



Classificador

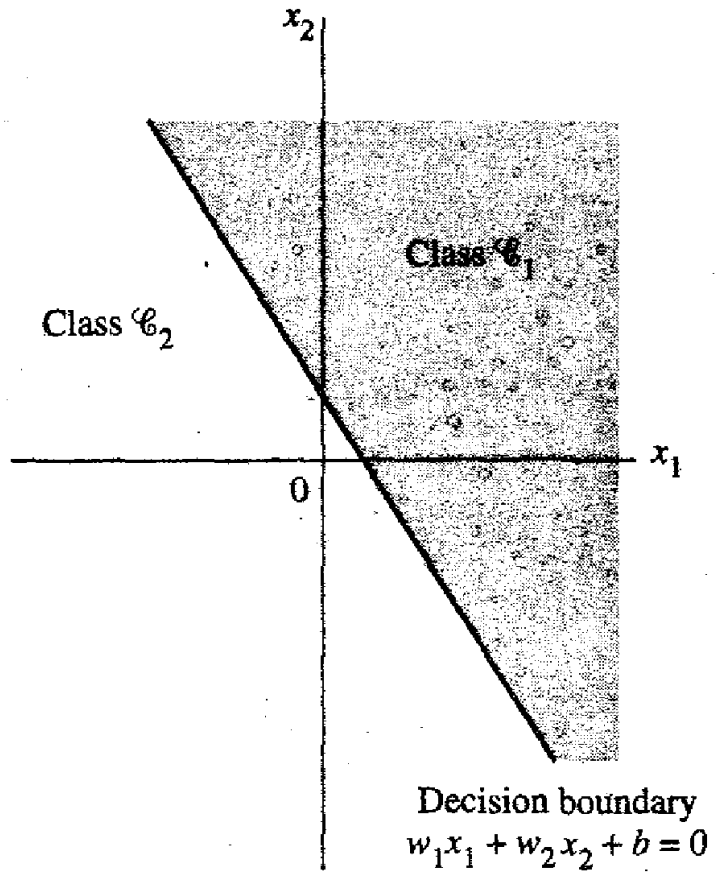
- O objetivo do perceptron é classificar o conjunto de estímulos $x_1, x_2, ..., x_m$ aplicados externamente, à classe $\mathscr{C}_1$ se a saída for $y = +1$ ou à classe $\mathscr{C}_2$ se a saída for -1.
- Na forma mais simples do perceptron, há duas regiões de decisão separadas por um hiperplano definido por

$$\sum_{i=1}^{m} w_i x_i + b = 0 \tag{2}$$

- Veja figura 1.

Hiperplano

Figure 1: Ilustração do hiperplano (nesse exemplo, uma linha reta) como fronteira de decisão para um problema de classificação de padrões de duas classes e duas dimensões [2].



Sumário

1. Perceptron
   - Histórico
   - O perceptron
   - Convergência do perceptron
2. Adaline
   - Adaptive Linear Element

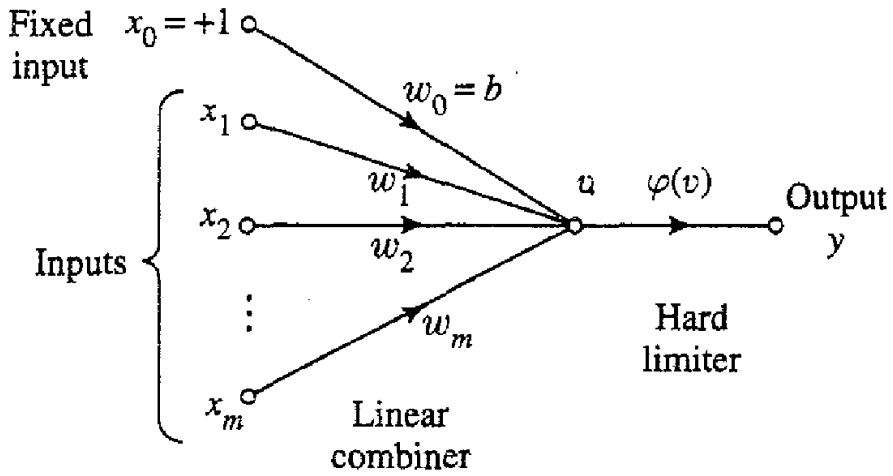
Correção de erros

- Para derivar o algoritmo de aprendizado por correção de erro para o perceptron, considere a figura abaixo [2].
- Vetor de entrada $(m+1)$ por 1:

$$\mathbf{x}(n) = [+1, x_1(n), x_2(n), ..., x_m(n)]^T \tag{3}$$

- Vetor de pesos $(m+1)$ por 1:

$$\mathbf{w}(n) = [b(n), w_1(n), w_2(n), ..., w_m(n)]^T \tag{4}$$

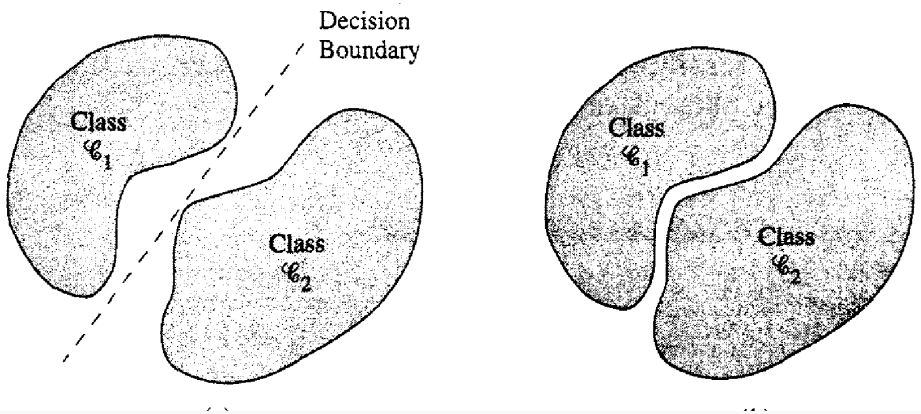


Combinador linear

- A saída do combinador linear é:

$$v(n) = \sum_{i=0}^{m} w_i(n)x_i(n) = \mathbf{w}^T(n)\mathbf{x}(n) \tag{5}$$

  onde $w_0(n)$ representa o *bias* $b(n)$.
- Para n fixo, a equação $\mathbf{w}^T\mathbf{x} = 0$, plotada num espaço m-dimensional define um hiperplano, como a superfície de decisão entre duas classes diferentes de entradas.
- Para o perceptron funcionar adequadamente, as duas classes $\mathscr{C}_1$ e $\mathscr{C}_2$ devem ser *linearmente separáveis* [2]:



Conjuntos de treinamento

- Seja $\mathscr{X}_1 = \mathbf{x}_1(1), \mathbf{x}_1(2), \ldots$ o subconjunto de vetores de treinamento que pertencem à classe $\mathscr{C}_1$ e $\mathscr{X}_2 = \mathbf{x}_2(1), \mathbf{x}_2(2), \ldots$ o subconjunto de vetores de treinamento que pertencem à classe $\mathscr{C}_2$.
- $\mathscr{X} = \mathscr{X}_1 \cup \mathscr{X}_2$.
- Dados os conjuntos $\mathscr{X}_1$ e $\mathscr{X}_2$ para treinar o classificador, o treinamento envolve o ajuste do vetor de pesos $\mathbf{w}$ tal que $\mathscr{C}_1$ e $\mathscr{C}_2$ sejam linearmente separáveis:
  - $\mathbf{w}^T\mathbf{x} > 0$ para todo vetor de entrada $\mathbf{x}$ pertencente à classe $\mathscr{C}_1$.
  - $\mathbf{w}^T\mathbf{x} \leq 0$ para todo vetor de entrada $\mathbf{x}$ pertencente à classe $\mathscr{C}_2$.
- O problema do treinamento é, dados $\mathscr{X}_1$ e $\mathscr{X}_2$, achar um vetor de pesos $\mathbf{w}$ tal que as desigualdades acima sejam satisfeitas.

Treinamento

- O algoritmo pode ser formulado assim:
  1. Se o n-ésimo termo do conjunto de treinamento $\mathbf{x}(n)$ é corretamente classificado por $\mathbf{w}(n)$ computado na n-ésima iteração do algoritmo, nenhuma correção é feita ao vetor de pesos de acordo com a regra
     1. $\mathbf{w}(n+1) = \mathbf{w}(n)$, se $\mathbf{w}^T\mathbf{x}(n) > 0$ e $\mathbf{x}(n)$ pertence à classe $\mathscr{C}_1$.
     2. $\mathbf{w}(n+1) = \mathbf{w}(n)$, se $\mathbf{w}^T\mathbf{x}(n) \leq 0$ e $\mathbf{x}(n)$ pertence à classe $\mathscr{C}_2$.
  2. Caso contrário, o vetor de pesos é atualizado de acordo com a regra
     1. $\mathbf{w}(n+1) = \mathbf{w}(n) - \eta(n)\mathbf{x}(n)$, se $\mathbf{w}^T(n)\mathbf{x}(n) > 0$ e $\mathbf{x}(n)$ pertence à classe $\mathscr{C}_2$.
     2. $\mathbf{w}(n+1) = \mathbf{w}(n) + \eta(n)\mathbf{x}(n)$, se $\mathbf{w}^T(n)\mathbf{x}(n) \leq 0$ e $\mathbf{x}(n)$ pertence à classe $\mathscr{C}_1$.

     onde $\eta(n)$ controla o ajuste aplicado ao vetor de pesos na iteração n.

Teorema da Convergência do Perceptron

- O *teorema da convergência de incremento fixo* para o perceptron pode ser enunciada:

  Sejam os subconjuntos dos vetores de treinamento $\mathscr{X}_1$ e $\mathscr{X}_2$ linearmente separáveis. Sejam as entradas apresentadas ao perceptron originárias desses dois subconjuntos. O perceptron converge depois de n_0 iterações, no sentido de que

$$\mathbf{w}(n_0) = \mathbf{w}(n_0 + 1) = \mathbf{w}(n_0 + 2) = ... \tag{6}$$

  seja um vetor solução para $n_0 \leq n_{max}$.

Teorema da Convergência do Perceptron

- Considere agora o *procedimento para correção de erro absoluto*, para o qual $\eta(n)$ é variável.
- Seja $\eta(n)$ o menor inteiro para o qual $\eta(n)\mathbf{x}^T\mathbf{x}(n) > |\mathbf{w}^T(n)\mathbf{x}(n)|$.
- Ou seja, se o produto interno $\mathbf{w}^T(n)\mathbf{x}(n)$ na iteração n tem um sinal incorreto, então $\mathbf{w}^T(n+1)\mathbf{x}(n)$ na iteração $n+1$ teria o sinal correto.
- Em outras palavras, cada padrão é apresentado repetidamente ao perceptron até que o padrão seja classificado corretamente.

Teorema da Convergência do Perceptron

- Sumário do Algoritmo de Convergência do Perceptron:
  - Variáveis e parâmetros:
    - Vetor de entrada: $\mathbf{x}(n) = [+1, x_1(n), x_2(n), ..., x_m(n)]^T$
    - Vetor de pesos: $\mathbf{w}(n) = [b(n), w_1(n), w_2(n), ..., w_m(n)]^T$
    - Bias: $b(n)$
    - Resposta real: $y(n)$
    - Resposta desejada: $d(n)$:

$$d(n) = \begin{cases} +1, & \text{se } \mathbf{x}(n) \text{ pertence a classe } \mathscr{C}_1 \\ -1, & \text{se } \mathbf{x}(n) \text{ pertence a classe } \mathscr{C}_2 \end{cases}$$

    - Taxa de aprendizado: η
    - Função *signum*:

$$sgn(v) = \begin{cases} +1, & \text{se } v > 0 \\ -1, & \text{se } v < 0 \end{cases}$$

Teorema da Convergência do Perceptron

- Sumário do Algoritmo de Convergência do Perceptron:
  1. *Iniciação*: Faça $\mathbf{w}(0) = 0$. Então realize as seguintes computações para o passo de tempo $n = 1, 2, ...$
  2. *Ativação*: No passo de tempo n, ative o perceptron aplicando os vetores de entradas contínuas $\mathbf{x}(n)$ e respostas desejadas $d(n)$
  3. *Computação da resposta real*: $y(n) = sgn[\mathbf{w}^T(n)\mathbf{x}(n)]$
  4. *Adaptação do vetor de pesos*: $\mathbf{w}(n+1) = \mathbf{w}(n) + \eta[d(n) - y(n)]\mathbf{x}(n)$
  5. *Continuação*: Incremente o passo de tempo n e vá ao passo 2.

Regra Delta

- Para um perceptron

$$w_i(n+1) = w_i(n) + \eta(d(n) - y(n)).x_i(n) \tag{7}$$

onde

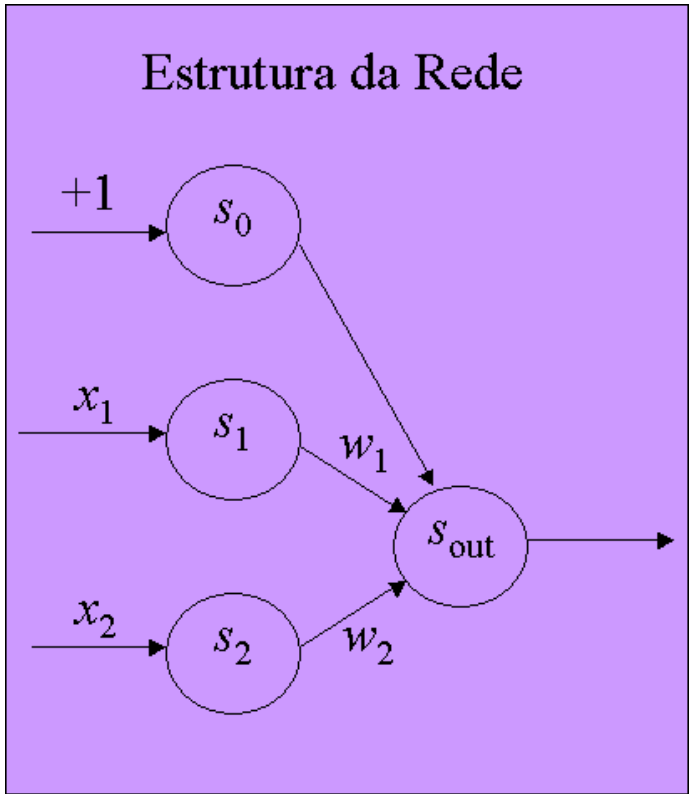
- η = taxa (ou coeficiente) de aprendizado
- $d(n)$ = saída desejada
- $y(n)$ = saída real
- $\delta = d(n) - y(n)$ = erro
- $x_i(n)$ = entrada i, $1 \leq i \leq m$

- Regra Delta:

$$\Delta w_i = w_i(n+1) - w_i(n) = \eta \delta x_i \tag{8}$$

Exemplo [6]

- Simulação do operador lógico AND:



Exemplo [6]

- Pesos iniciais: $w_0 = 0,\ w_1 = 0,\ w_2 = 0.$
- Taxa de aprendizado: $\eta = 0.5.$

AND	x_0	x_1	x_2	t
Entrada 1:	1	0	0	0
Entrada 2:	1	0	1	0
Entrada 3:	1	1	0	0
Entrada 4:	1	1	1	1

Exemplo [6]

- 1° Ciclo
  - Entrada 1: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(0 \times 1 + 0 \times 0 + 0 \times 0) = f(0) = 0 \longrightarrow s_{out} = t$
  - Entrada 2: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(0 \times 1 + 0 \times 1 + 0 \times 0) = f(0) = 0 \longrightarrow s_{out} = t$
  - Entrada 3: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(0 \times 1 + 0 \times 0 + 0 \times 1) = f(0) = 0 \longrightarrow s_{out} = t$
  - Entrada 4: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(0 \times 1 + 0 \times 1 + 0 \times 1) = f(0) = 0 \longrightarrow s_{out} \neq t$
  - Pesos:
    - $w_0 = w_0 + \eta(t - s_{out})x_0 = 0 + 0.5 \times (1 - 0) \times 1 = 0.5$
    - $w_1 = w_1 + \eta(t - s_{out})x_1 = 0 + 0.5 \times (1 - 0) \times 1 = 0.5$
    - $w_2 = w_2 + \eta(t - s_{out})x_2 = 0 + 0.5 \times (1 - 0) \times 1 = 0.5$

Exemplo [6]

- 2° Ciclo
  - Entrada 1: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(0.5 \times 1 + 0.5 \times 0 + 0.5 \times 0) = f(0.5) = 1 \longrightarrow s_{out} \neq t$
  - Pesos:
    - $w_0 = w_0 + \eta(t - s_{out})x_0 = 0.5 + 0.5 \times (0 - 1) \times 1 = 0$
    - $w_1 = w_1 + \eta(t - s_{out})x_1 = 0.5 + 0.5 \times (0 - 1) \times 0 = 0.5$
    - $w_2 = w_2 + \eta(t - s_{out})x_2 = 0.5 + 0.5 \times (0 - 1) \times 0 = 0.5$
  - Entrada 2: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(0 \times 1 + 0.5 \times 0 + 0.5 \times 1) = f(0.5) = 1 \longrightarrow s_{out} \neq t$
  - Pesos:
    - $w_0 = w_0 + \eta(t - s_{out})x_0 = 0 + 0.5 \times (0 - 1) \times 1 = -0.5$
    - $w_1 = w_1 + \eta(t - s_{out})x_1 = 0.5 + 0.5 \times (0 - 1) \times 0 = 0.5$
    - $w_2 = w_2 + \eta(t - s_{out})x_2 = 0.5 + 0.5 \times (0 - 1) \times 1 = 0$

Exemplo [6]

- 2° Ciclo (cont.)
  - Entrada 3: $s_{out} = f(w_0 x_0 + w_1 x_1 + w_2 x_2) = f(-0.5 \times 1 + 0.5 \times 1 + 0 \times 0) = f(0) = 0 \longrightarrow s_{out} = t$
  - Entrada 4: $s_{out} = f(w_0 x_0 + w_1 x_1 + w_2 x_2) = f(-0.5 \times 1 + 0.5 \times 1 + 0 \times 1) = f(0) = 0 \longrightarrow s_{out} \neq t$
  - Pesos:
    - $w_0 = w_0 + \eta(t - s_{out})x_0 = -0.5 + 0.5 \times (1 - 0) \times 1 = 0$
    - $w_1 = w_1 + \eta(t - s_{out})x_1 = 0.5 + 0.5 \times (1 - 0) \times 1 = 1$
    - $w_2 = w_2 + \eta(t - s_{out})x_2 = 0 + 0.5 \times (1 - 0) \times 1 = 0.5$

Exemplo [6]

- 3° Ciclo
  - Entrada 1: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(0 \times 1 + 1 \times 0 + 0.5 \times 0) = f(0) = 0 \longrightarrow s_{out} = t$
  - Entrada 2: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(0 \times 1 + 1 \times 0 + 0.5 \times 1) = f(0.5) = 1 \longrightarrow s_{out} \neq t$
  - Pesos:
    - $w_0 = w_0 + \eta(t - s_{out})x_0 = 0 + 0.5 \times (0 - 1) \times 1 = -0.5$
    - $w_1 = w_1 + \eta(t - s_{out})x_1 = 1 + 0.5 \times (0 - 1) \times 0 = 1$
    - $w_2 = w_2 + \eta(t - s_{out})x_2 = 0.5 + 0.5 \times (0 - 1) \times 1 = 0$
  - Entrada 3: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(-0.5 \times 1 + 1 \times 1 + 0 \times 0) = f(0.5) = 1 \longrightarrow s_{out} \neq t$
  - Pesos:
    - $w_0 = w_0 + \eta(t - s_{out})x_0 = -0.5 + 0.5 \times (0 - 1) \times 1 = -1$
    - $w_1 = w_1 + \eta(t - s_{out})x_1 = 1 + 0.5 \times (0 - 1) \times 1 = 0.5$
    - $w_2 = w_2 + \eta(t - s_{out})x_2 = 0 + 0.5 \times (0 - 1) \times 0 = 0$

Exemplo [6]

- 3° Ciclo (cont.)
  - Entrada 4: $s_{out} = f(w_0 x_0 + w_1 x_1 + w_2 x_2) = f(-1 \times 1 + 0.5 \times 1 + 0 \times 1) = f(-0.5) = 0 \longrightarrow s_{out} \neq t$
  - Pesos:
    - $w_0 = w_0 + \eta(t - s_{out})x_0 = -1 + 0.5 \times (1 - 0) \times 1 = -0.5$
    - $w_1 = w_1 + \eta(t - s_{out})x_1 = 0.5 + 0.5 \times (1 - 0) \times 1 = 1$
    - $w_2 = w_2 + \eta(t - s_{out})x_2 = 0 + 0.5 \times (1 - 0) \times 1 = 0.5$

Exemplo [6]

- 4° Ciclo
  - Entrada 1: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(-0.5 \times 1 + 1 \times 0 + 0.5 \times 0) = f(-0.5) = 0 \longrightarrow s_{out} = t$
  - Entrada 2: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(-0.5 \times 1 + 1 \times 0 + 0.5 \times 1) = f(0) = 0 \longrightarrow s_{out} = t$
  - Entrada 3: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(-0.5 \times 1 + 1 \times 1 - 0.5 \times 0) = f(0.5) = 1 \longrightarrow s_{out} \neq t$
  - Pesos:
    - $w_0 = w_0 + \eta(t - s_{out})x_0 = -0.5 + 0.5 \times (0 - 1) \times 1 = -1$
    - $w_1 = w_1 + \eta(t - s_{out})x_1 = 1 + 0.5 \times (0 - 1) \times 1 = 0.5$
    - $w_2 = w_2 + \eta(t - s_{out})x_2 = 0.5 + 0.5 \times (0 - 1) \times 0 = 0.5$
  - Entrada 4: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(-1 \times 1 + 0.5 \times 1 + 0.5 \times 1) = f(0) = 0 \longrightarrow s_{out} \neq t$
  - Pesos:
    - $w_0 = w_0 + \eta(t - s_{out})x_0 = -1 + 0.5 \times (1 - 0) \times 1 = -0.5$
    - $w_1 = w_1 + \eta(t - s_{out})x_1 = 0.5 + 0.5 \times (1 - 0) \times 1 = 1$
    - $w_2 = w_2 + \eta(t - s_{out})x_2 = 0.5 + 0.5 \times (1 - 0) \times 1 = 1$

Exemplo [6]

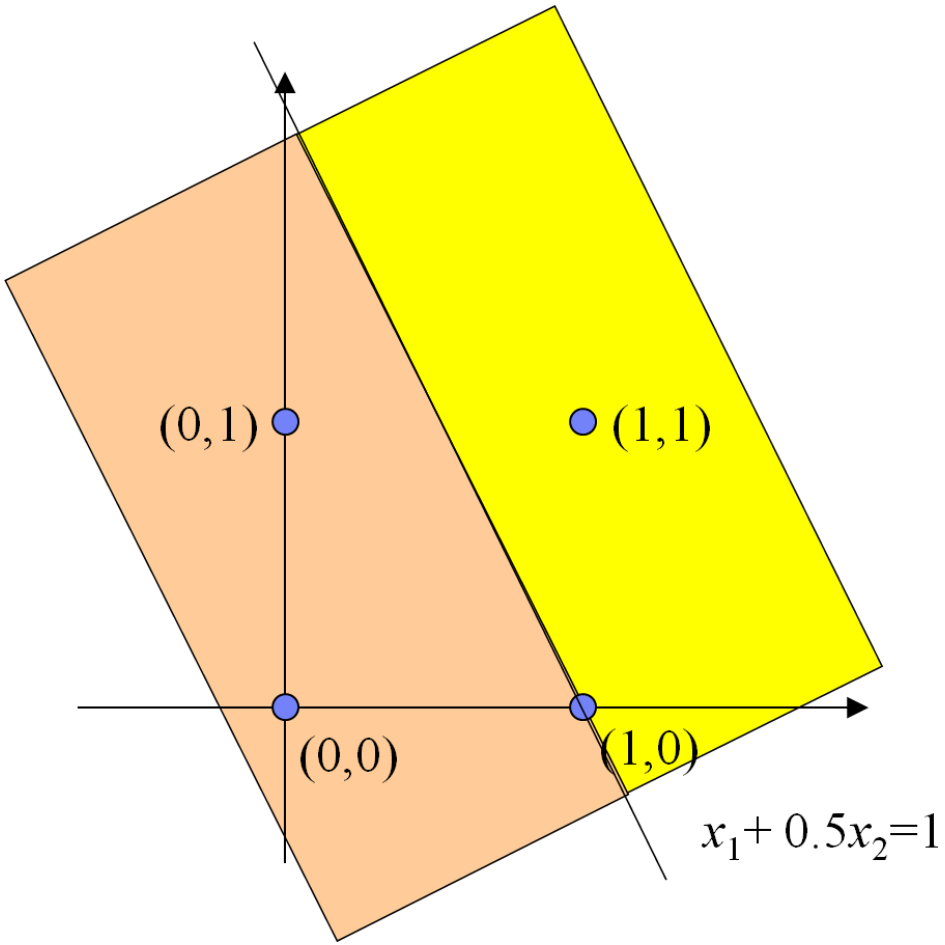
- 5° Ciclo
  - Entrada 1: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) =$
    $f(-0.5 \times 1 + 1 \times 0 + 1 \times 0) = f(-0.5) = 0 \longrightarrow s_{out} = t$
  - Entrada 2: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) =$
    $f(-0.5 \times 1 + 1 \times 0 + 1 \times 1) = f(0.5) = 1 \longrightarrow s_{out} \neq t$
  - Pesos:
    - $w_0 = w_0 + \eta(t - s_{out})x_0 = -0.5 + 0.5 \times (0 - 1) \times 1 = -1$
    - $w_1 = w_1 + \eta(t - s_{out})x_1 = 1 + 0.5 \times (0 - 1) \times 0 = 1$
    - $w_2 = w_2 + \eta(t - s_{out})x_2 = 1 + 0.5 \times (0 - 1) \times 1 = 0.5$
  - Entrada 3: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) =$
    $f(-1 \times 1 + 1 \times 1 + 0.5 \times 0) = f(0) = 0 \longrightarrow s_{out} = t$
  - Entrada 4: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) =$
    $f(-1 \times 1 + 1 \times 1 + 0.5 \times 1) = f(0.5) = 1 \longrightarrow s_{out} = t$

Exemplo [6]

- 6° Ciclo
  - Entrada 1: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(-1 \times 1 + 1 \times 0 + 0.5 \times 0) = f(-1) = 0 \longrightarrow s_{out} = t$
  - Entrada 2: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(-1 \times 1 + 1 \times 0 + 0.5 \times 1) = f(-0.5) = 0 \longrightarrow s_{out} = t$
  - Entrada 3: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(-1 \times 1 + 1 \times 1 + 0.5 \times 0) = f(0) = 0 \longrightarrow s_{out} = t$
  - Entrada 4: $s_{out} = f(w_0x_0 + w_1x_1 + w_2x_2) = f(-1 \times 1 + 1 \times 1 + 0.5 \times 1) = f(0.5) = 1 \longrightarrow s_{out} = t$
  - Pesos Finais (**solução**):
    - $w_0 = -1$
    - $w_1 = 1$
    - $w_2 = 0.5$

Exemplo [6]: Interpretação geométrica

- *Linha de Decisão*: $x_1 w_1 + x_2 w_2 = -\theta \Longrightarrow x_1 + 0.5 x_2 = 1.$



Sumário

1. Perceptron
   - Histórico
   - O perceptron
   - Convergência do perceptron
2. Adaline
   - Adaptive Linear Element

O Adaline [8]

- O adaline foi proposto em 1960 por Widrow e Hoff [9].
- *Introduziu* a regra delta.
- Seu aprendizado é fundamentado na regra delta (algoritmo LMS).
- O adaline também é constituído de apenas uma camada neural.
- Vários adalines formando uma rede é chamado de *Madaline*.

O Adaline

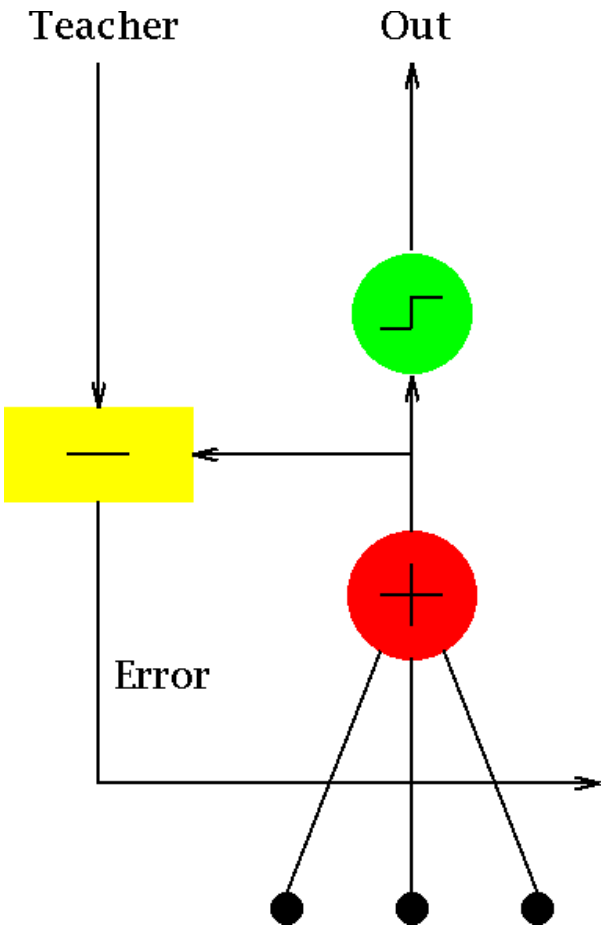


Figure 2: Adaline [5]

O Adaline

- O adaline é muito parecido com o perceptron.
- Diferença [1]:
  - O perceptron é um separador linear e é usado em classificação de padrões,
  - O adaline é um aproximador linear de funções e é usado em aproximação de funções.

Referências I

[1] A. P. Braga, A. P. L. F. Carvalho e T. B. Ludermir
Redes Neurais Artificiais - Teoria e Aplicações.
2a. edição. Editora LTC, 2007.

[2] S. Haykin
Neural networks - a comprehensive foundation.
2nd. edition. Prentice Hall, 1999.

[3] D. O. Hebb
The Organization of Behavior: A Neuropsychological Theory.
Wiley, 1949.

[4] W. S. McCulloch and W. Pitts
A logical calculus of the ideas immanent in nervous activity
Bulletin of Mathematical Biophysics, 5, pp. 115-133, 1943.

Referências II

[5] Paskari (talk) (Uploads) - Own work
"Adaline". Licensed under Public Domain via Wikipedia
http://en.wikipedia.org/wiki/File:Adaline.gif

[6] R. A. F. Romero
SCC-5809 Redes Neurais.
Slides e listas de exercícios. Programa de Pós-Graduação em Ciência de Computação e Matemática Computacional. ICMC/USP, 2010.

[7] F. Rosenblatt
The Perceptron: A probabilistic model for information storage and organization in the brain.
Psychological Review, vol. 65, pp. 386–408, 1958.

Referências III

[8] I. N. da Silva, D. H. Spatti, e R. A. Flauzino,
Redes Neurais Artificiais para engenharia e ciências aplicadas - curso prático.
Artliber Editora, 2010.

[9] B. Widrow and M. E. Hoff
"Adaptive switching circuits."
in *1960 IRE WESCON Convention Record*, pp. 96–104, New York, 1960.